The Board of Directors Debate

How the multi-agent debate works in VENTUREGENESIS

Code walkthrough • debate_engine.py and agents/board.py

The two layers

There are two distinct things named “board” in this codebase, and they nest together.

The **debate engine** (`debate_engine.py`) is the actual multi-agent argument — six role-playing agents talking across three rounds. The **board agent** (`agents/board.py`) is the wrapper: it runs the whole analysis pipeline (ML models, intelligence agents, and the debate), then has a Chairperson synthesize one final decision. The debate is just one of the inputs the Chairperson reads.

The debate itself (debate_engine.py)

Six fixed roles sit on the board: **Founder, Investor, Financial, Customer, Market, and Competitor**. They move through three rounds, and the structure of each round is defined entirely by prompts in `prompts.yaml`.

Round 1 — independent opinions

Each agent receives the startup context and gives its own take in 2–3 sentences, with a stance of *bullish*, *neutral*, or *bearish* (the `debate_round_1` prompt). No agent has seen any other yet.

Round 2 — challenges

Each agent is now fed the prior transcript and told to push back on, or build on, at least one other agent’s position (`debate_round_2`).

Round 3 — consensus

Agents see the debate so far and move toward agreement, each giving a `final_recommendation` (`debate_round_3`).

The loop lives in `run_debate()`. For each round it builds the `prior` string from the last six messages, then calls every role in turn via `_agent_turn()`, which picks the right prompt for the round and calls the live LLM (`llm.complete`). Each agent’s turn is wrapped in try/except — if one agent errors, the engine emits an `agent_skipped` event and the debate continues rather than aborting.

How consensus is decided

It is **not** the LLM that declares a winner. After round 3, the code takes a plain majority vote on the stances from round-3 messages only, then picks whichever stance appears most often:

```
stances = [m["stance"] for m in transcript if m["round"] == 3]
consensus = max(set(stances), key=stances.count) if stances else "neutral"
```

Two ways it runs

`run_debate()` is a generator that **yields events** (`start`, `round_start`, `message`, `round_end`, `consensus`). The `sse_format()` helper wraps each event as Server-Sent Events, so the API can stream the debate to the frontend live, one message at a time.

`run_debate_blocking()` is the synchronous version — it drains the generator and returns just `{consensus, transcript}`. This is the variant the board agent calls.

One detail worth flagging

`_stance_for()` contains hardcoded heuristic stances based on runway, growth, and churn — e.g. Competitor is always bearish, and Financial turns bearish when runway is under 8 months. But note it is **defined and never called** in the current flow; the real stances come from the LLM responses. It looks like a leftover fallback. If stances were meant to be grounded in the metrics rather than left entirely to the model, that function is the unused hook for it.

How the board consumes it

In `board.py`, `gather_all()` runs the debate as one of roughly fourteen agents, then `_compact()` extracts only `debate_consensus` into the trimmed summary handed to the Chairperson. So the multi-round argument collapses to a single consensus label by the time the final `INVEST | HOLD | PIVOT | WIND_DOWN` decision is made.

At a glance

Stage	What happens	Who decides
Round 1	Independent opinions, 2-3 sentences + stance	Each of 6 agents (LLM)
Round 2	Challenge / build on others' positions	Each agent, sees prior transcript
Round 3	Move to consensus + final recommendation	Each agent
Consensus	Majority vote over round-3 stances	Code (max-count), not LLM
Board decision	INVEST / HOLD / PIVOT / WIND_DOWN	Chairperson (LLM) over all agents